TP1 nosql module

Hasna Daoui

April 2026

1 Mini-TP 3 : Démarrage Docker & MongoDB

Objectif

Lancer un conteneur MongoDB et vérifier son bon fonctionnement.

Prérequis

Docker installé et fonctionnel (`docker ps` ne doit pas renvoyer d'erreur).

Étape 1 — Lancer le conteneur MongoDB

```
docker run -d \
  --name mongo-noseclab \
  -p 27017:27017 \
  -v mongo-data:/data/db \
  mongo:7
```

Windows (PowerShell) :

```
docker run -d --name mongo-noseclab -p 27017:27017 -v mongo-data:/data/db mongo:7
```

Étape 2 — Vérifier que le conteneur tourne

```
docker ps
```

Résultat attendu : statut `Up`.

Étape 3 — Vérifier les logs

```
docker logs mongo-noseclab | tail -5
```

Résultat attendu : présence de `"Waiting for connections"`.

Étape 4 — Lancer mongosh

```
docker exec -it mongo-noseclab mongosh
```

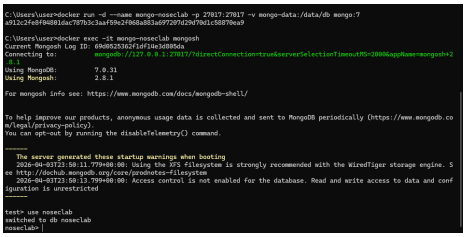
Résultat attendu : prompt `test>`.

Étape 5 — Test rapide

```
db.runCommand({ ping: 1 })
```

Résultat attendu :

```
{ ok: 1 }
```



2 Mini-TP 4 : Création de la base & insertion du dataset security_logs

Objectif

Créer la base `noseclab`, la collection `security_logs`, et insérer un dataset.

Prérequis

Conteneur MongoDB en cours d'exécution.

Étape 1 — Connexion à mongosh

```
docker exec -it mongo-noseclab mongosh
```

Étape 2 — Création de la base

```
use noseclab
```

Étape 3 — Insertion du dataset

```
db.security_logs.insertMany([
  {
    timestamp: ISODate("2026-02-25T08:01:00Z"),
    src_ip: "192.168.1.10",
    user: "alice",
    action: "login",
    status: "success",
    device: "laptop",
    geo: { country: "FR", city: "Paris" },
    severity: "info"
  },
  {
    timestamp: ISODate("2026-02-25T08:05:23Z"),
    src_ip: "192.168.1.10",
    user: "alice",
    action: "file_access",
    status: "success",
    device: "laptop",
    geo: { country: "FR", city: "Paris" },
    severity: "info"
  }
])
```

Étape 4 — Vérifications

```
// Bases
show dbs

// Collections
show collections

// Nombre de documents
db.security_logs.countDocuments()

// Aperçu
db.security_logs.findOne()
```

Résultats attendus

- `noseclab` apparaît dans `show dbs`
- `security_logs` apparaît dans `show collections`
- `countDocuments()` retourne 12
- `findOne()` retourne un document avec un `_id`

Auto-check

- 12 documents insérés
- `_id` auto-généré présent
- Documents flexibles (ex: port_scan sans `user`)

Dépannage

```
// Redémarrer MongoDB si erreur
docker restart mongo-noseclab
```

```
|   {
|     timestamp: ISODate("2026-02-25T11:00:00Z"),
|     src_ip: "192.168.1.25",
|     user: "charlie",
|     action: "login",
|     status: "success",
|     device: "server-web01",
|     geo: { country: "FR", city: "Marseille" },
|     severity: "info"
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d054d262f1df14e3d805e1'),
    '1': ObjectId('69d054d262f1df14e3d805e2'),
    '2': ObjectId('69d054d262f1df14e3d805e3'),
    '3': ObjectId('69d054d262f1df14e3d805e4'),
    '4': ObjectId('69d054d262f1df14e3d805e5'),
    '5': ObjectId('69d054d262f1df14e3d805e6'),
    '6': ObjectId('69d054d262f1df14e3d805e7'),
    '7': ObjectId('69d054d262f1df14e3d805e8'),
    '8': ObjectId('69d054d262f1df14e3d805e9'),
    '9': ObjectId('69d054d262f1df14e3d805ea'),
    '10': ObjectId('69d054d262f1df14e3d805eb'),
    '11': ObjectId('69d054d262f1df14e3d805ec')
  }
}
noseclab>
```

```
noseclab> show dbs
admin     40.00 KiB
config    60.00 KiB
local     40.00 KiB
noseclab  72.00 KiB
noseclab> show collections
security_logs
noseclab> db.security_logs.countDocuments()
12
noseclab> db.security_logs.findOne()
{
  _id: ObjectId('69d055a862f1df14e3d805ed'),
  timestamp: ISODate('2026-02-25T08:01:00.000Z'),
  src_ip: '192.168.1.10',
  user: 'alice',
  action: 'login',
  status: 'success',
  device: 'laptop',
  geo: { country: 'FR', city: 'Paris' },
  severity: 'info'
}
noseclab>
```

3 Mini-TP 5 : Exercices CRUD guidés

Objectif

Maîtriser les opérations CRUD sur la collection `security_logs`.

Prérequis

Base `noseclab` avec 12 documents existants.

Exercice A — Create

```
db.security_logs.insertOne({
  timestamp: ISODate("2026-02-25T11:45:00Z"),
  src_ip: "198.51.100.7",
  user: "eve",
  action: "login",
  status: "failure",
  device: "smartphone",
  geo: { country: "US", city: "New York" },
  severity: "warning"
})
```

Vérification :

```
db.security_logs.countDocuments()
```

Résultat attendu : 13

Exercice B — Read

```
// B1
db.security_logs.find({ status: "failure" })

// B2
db.security_logs.find({ user: "bob" })

// B3
db.security_logs.find({ severity: "critical" })

// B4
db.security_logs.find({ "geo.country": "CN" })
```

Exercice C — Update

```
// C1
db.security_logs.updateOne(
  { src_ip: "172.16.0.99", action: "port_scan" },
```

```
  { $set: { investigated: true, severity: "info" } }
)

// C2
db.security_logs.updateMany(
  { action: "login", status: "failure" },
  { $set: { severity: "high" } }
)
```

Exercice D — Delete

```
db.security_logs.deleteOne({
  user: "eve",
  src_ip: "198.51.100.7"
})
```

Vérification :

```
db.security_logs.countDocuments()
```

Résultat attendu : 12

Attention

```
db.security_logs.deleteMany({})
db.security_logs.drop()
```

Auto-check

- 12 documents après suppression
- Champ `investigated: true` présent sur port_scan
- Tous les `login failure` ont `severity: "high"`

4 Mini-TP 6 : Scénarios de requêtes cybersécurité

Objectif

Utiliser MongoDB pour répondre à des questions d'investigation sécurité.

Prérequis

Collection `security_logs` avec 12 documents.

```
noseclab> db.security_logs.insertOne({
|   timestamp: ISODate("2026-02-25T11:45:00Z"),
|   src_ip: "198.51.100.7",
|   user: "eve",
|   action: "login",
|   status: "failure",
|   device: "smartphone",
|   geo: { country: "US", city: "New York" },
|   severity: "warning"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69d0566062f1df14e3d805f9')
}
noseclab> db.security_logs.countDocuments()
13
noseclab> db.security_logs.find({ status: "failure" })
[
  {
    _id: ObjectId('69d055a862f1df14e3d805ef'),
    timestamp: ISODate('2026-02-25T08:12:45.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'warning'
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f0'),
    timestamp: ISODate('2026-02-25T08:13:10.000Z'),
```

```
noseclab> db.security_logs.find({ user: "bob" })
[
  {
    _id: ObjectId('69d055a862f1df14e3d805ef'),
    timestamp: ISODate('2026-02-25T08:12:45.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'warning'
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f0'),
    timestamp: ISODate('2026-02-25T08:13:10.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'warning'
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f1'),
    timestamp: ISODate('2026-02-25T08:13:42.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
```

```
noseclab> db.security_logs.find({ severity: "critical" })
[
  {
    _id: ObjectId('69d055a862f1df14e3d805f3'),
    timestamp: ISODate('2026-02-25T09:00:00.000Z'),
    src_ip: '172.16.0.99',
    action: 'port_scan',
    status: 'detected',
    severity: 'critical'
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f7'),
    timestamp: ISODate('2026-02-25T10:30:00.000Z'),
    src_ip: '192.168.1.25',
    user: 'charlie',
    action: 'privilege_escalation',
    status: 'failure',
    device: 'server-web01',
    geo: { country: 'FR', city: 'Marseille' },
    severity: 'critical'
  }
]
noseclab> db.security_logs.find({ "geo.country": "CN" })
[
  {
    _id: ObjectId('69d055a862f1df14e3d805f5'),
    timestamp: ISODate('2026-02-25T10:00:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
    geo: { country: 'CN', city: 'Beijing' },
    severity: 'high'
```

Scénario 1 — Détection de brute-force

```
db.security_logs.find({
  user: "bob",
  action: "login",
  status: "failure"
})
```

```
noseclab> db.security_logs.updateOne(
|   { src_ip: "172.16.0.99", action: "port_scan" },
|   { $set: { investigated: true, severity: "info" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
noseclab> db.security_logs.updateMany(
|   { action: "login", status: "failure" },
|   { $set: { severity: "high" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 3,
  upsertedCount: 0
}
noseclab> db.security_logs.deleteOne({
|   user: "eve",
|   src_ip: "198.51.100.7"
| })
{ acknowledged: true, deletedCount: 1 }
noseclab> db.security_logs.countDocuments()
12
noseclab> |
```

Résultat attendu : 3 documents (même IP, intervalle court).

Scénario 2 — Activité depuis l'étranger

```
db.security_logs.find({
  "geo.country": { $ne: "FR" }
})
```

Note : Les documents sans champ `geo` sont aussi retournés.

Scénario 3 — 3 derniers événements critiques

```
db.security_logs.find(
  { severity: { $in: ["high", "critical"] } },
  { timestamp: 1, src_ip: 1, action: 1, severity: 1, _id: 0 }
).sort({ timestamp: -1 }).limit(3)
```

Scénario 4 — Nombre d'événements par action

```
db.security_logs.aggregate([
  { $group: { _id: "$action", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Scénario 5 — Exercice libre (solution)

```
db.security_logs.find(
  { action: "login", status: "success" },
  { user: 1, device: 1, timestamp: 1, _id: 0 }
).sort({ timestamp: 1 })
```

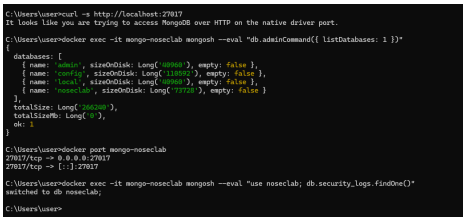
Auto-check

- Scénario 1 : 3 documents pour `bob`
- Scénario 3 : seulement 4 champs retournés (sans `_id`)
- Scénario 2 : inclut les documents sans `geo`

Note technique

```
// Exclure les documents sans champ geo
db.security_logs.find({
  "geo.country": { $exists: true, $ne: "FR" }
})
```

```
noseclab> db.security_logs.find({
|   "geo.country": { $ne: "FR" }
| }).toArray()
[
  {
    _id: ObjectId('69d055a862f1df14e3d805f3'),
    timestamp: ISODate('2026-02-25T09:00:00.000Z'),
    src_ip: '172.16.0.99',
    action: 'port_scan',
    status: 'detected',
    severity: 'info',
    investigated: true
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f5'),
    timestamp: ISODate('2026-02-25T10:00:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
    geo: { country: 'CN', city: 'Beijing' },
    severity: 'high'
  },
  {
    _id: ObjectId('69d055a862f1df14e3d805f6'),
    timestamp: ISODate('2026-02-25T10:01:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
    geo: { country: 'CN', city: 'Beijing' },
    severity: 'high'
  }
]
noseclab> |
```



```
C:\Users\user>curl -s http://localhost:27017
It looks like you are trying to access MongoDB over HTTP on the native driver port.

C:\Users\user>docker exec -it mongo-noseclab mongosh --eval "db.adminCommand({ listDatabases: 1 })"
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('110592'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'noseclab', sizeOnDisk: Long('737280'), empty: false }
  ],
  totalSize: Long('266240'),
  totalSizeMb: Long('0'),
  ok: 1
}

C:\Users\user>docker port mongo-noseclab
27017/tcp -> 0.0.0.0:27017
27017/tcp -> [::]:27017

C:\Users\user>docker exec -it mongo-noseclab mongosh --eval "use noseclab; db.security_logs.findOne()"
switched to db noseclab;

C:\Users\user>
```

```
noseclab> db.security_logs.aggregate([
|   { $group: { _id: "$action", count: { $sum: 1 } } },
|   { $sort: { count: -1 } }
| ]).toArray()
|
[
  { _id: 'login', count: 8 },
  { _id: 'privilege_escalation', count: 1 },
  { _id: 'logout', count: 1 },
  { _id: 'file_access', count: 1 },
  { _id: 'port_scan', count: 1 }
]
noseclab> db.security_logs.find(
|   { action: "login", status: "success" },
|   { user: 1, device: 1, timestamp: 1, _id: 0 }
| ).sort({ timestamp: 1 }).toArray()
[
  {
    timestamp: ISODate('2026-02-25T08:01:00.000Z'),
    user: 'alice',
    device: 'laptop'
  },
  {
    timestamp: ISODate('2026-02-25T08:15:00.000Z'),
    user: 'bob',
    device: 'desktop'
  },
  {
    timestamp: ISODate('2026-02-25T11:00:00.000Z'),
    user: 'charlie',
    device: 'server-web01'
  }
]
noseclab>
```

Livrable : Audit de la surface d'attaque MongoDB

1. Vérification de l'exposition réseau

L'audit a été réalisé sur le conteneur `mongo-noseclab` pour évaluer la visibilité du service sur le réseau.

[language=bash, caption=Vérification du port ouvert (Nmap)] nmap -p 27017 localhost Résultat : 27017/tcp open mongod

[language=bash, caption=Vérification du binding Docker] docker port mongo-noseclab Résultat : 0.0.0.0:27017 -¿ 27017/tcp

2. Checklist Sécurité — Séance 1

Le tableau suivant résume l'état de sécurité actuel de l'instance déployée :

Point de contrôle	**Commande utilisée**	**Résultat**
Port 27017 ouvert ?	`nmap` / `Test-NetConnection`	**OUI**
Authentification activée ?	`mongosh --eval`	**NON**
Interface de Binding	`docker port`	**0.0.0.0** (Toutes)
Accès sans mot de passe ?	`show dbs`	**OUI**
Lecture des logs possible ?	`db.security_logs.find()`	**OUI**

3. Analyse des risques et mesures correctives

Risques identifiés

1. **Exfiltration de données :** Un attaquant distant peut copier l'intégralité de la base `noseclab` sans laisser de traces d'authentification.

2. **Attaque par Ransomware :** Suppression des données originales et remplacement par une demande de rançon (attaque classique sur les instances MongoDB exposées).

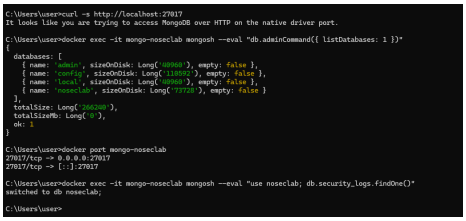
3. **Injection / Manipulation :** Modification des logs de sécurité pour masquer une intrusion ou compromettre l'intégrité de l'audit.

Mesures correctives recommandées

Restriction du binding : Limiter l'accès à l'interface locale (`127.0.0.1`) dans la configuration Docker.

Authentification RBAC : Activer le mode `--auth` et créer des utilisateurs avec des privilèges restreints (moindre privilège).

Chiffrement des flux : Implémenter TLS/SSL pour protéger les données en transit contre l'interception (Man-in-the-Middle).



```
C:\Users\user>curl -s http://localhost:27017
It looks like you are trying to access MongoDB over HTTP on the native driver port.

C:\Users\user>docker exec -it mongo-noseclab mongosh --eval "db.adminCommand({ listDatabases: 1 })"
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('110592'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'noseclab', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('266240'),
  totalSizeMb: Long('0'),
  ok: 1
}

C:\Users\user>docker port mongo-noseclab
27017/tcp -> 0.0.0.0:27017
27017/tcp -> [::]:27017

C:\Users\user>docker exec -it mongo-noseclab mongosh --eval "use noseclab; db.security_logs.findOne()"
switched to db noseclab;

C:\Users\user>
```